```
When /^I register for the Frequent Flyer program$/ do
  ...
end
```

If you prefer sticking to the JVM ecosystem, the following would be the equivalent of the previous step implementation using Cucumber-JVM and Groovy:[3]

```
@When(~"I register for the Frequent Flyer program") {
  ...
}
```

No matter which tool you're using, step definitions are usually global–you can't have the same text referring to different methods in different scenarios, although at times this might be useful.

6.2.3 Passing parameters to step implementations

Simple step definitions like the preceding ones won't get you very far. To automate scenarios effectively, you need to be able to extract data from the step definitions so that you can use them in the implementation code. Fortunately, BDD tools make it very easy to do this.

Suppose you need to automate the following three steps from different scenarios:

```
Given I am a Bronze Frequent Flyer member
Given I am a Silver Frequent Flyer member
Given I am a Gold Frequent Flyer member
```

These all represent effectively the same action (setting up a Frequent Flyer member with a given status). It would clearly be inefficient if you had to write a separate step implementation for each one of these variations. Fortunately, there's an alternative. All of the BDD tools let you pass variables into your step implementations—both individual field values and entire tables of data. For example, in Ruby, you could parameterize the previous steps like this:

```
Given /^I am a (.*) Frequent Flyer member$/ do | status |
    ...
end
```

This would pass the Frequent Flyer status value in to the `status` parameter, so that it could be used in the step implementation.

BDD tools that use typed languages can also often take advantage of automatic type conversions when the parameters are passed in to the step implementations. For example, here's the same step implementation, implemented using Cucumber-JVM in Java, which uses an enumeration to define the different possible status values:

---

[3] Groovy is a dynamic JVM language with a syntax similar to Java but with many features usually found in more flexible languages such as Ruby and Python.